

Laboratory assignment

Component 4

Authors: Moga Patricia, Ciobanu Sergiu-Tudor

Group: 5

May 28, 2026

1 Problem Definition

The aim of this project is the design and implementation of a Multi-Agent System that autonomously plays Ultimate Tic-Tac-Toe, a complex, hierarchical version of the classic game. The environment consists of a global 3×3 grid, where each cell contains a local 3×3 board. The primary challenge for the agents is navigating the catch in the rules: a player's move is a board determines the specific local board in which the opponent must play next. The system must solve the problem of strategic planning under constraints, where agents compete to win three local boards in a row while simultaneously managing the board-redirection mechanic to prevent the opponent from gaining an advantage. This simulation will be used to observe the competition between autonomous entities and determine if a first-mover advantage exists in this extended state space.

2 High-level MAS Specification

2.1 Inputs and Outputs to/from the MAS system

- **Inputs:** The inputs to the system consist of the complete state of the 9×9 game board, the current status of each of the nine local boards (Won, Lost, or Draw), and the coordinates of the last move which dictate the "active" local board for the current turn
- **Outputs:** The output of the MAS is the specific coordinate pair (x, y) chosen by the active agent for its move, which is then updated on the global board state. Agents also output their internal evaluation scores to help visualize the advantage level of each player.

2.2 Types of Agents

The system utilizes two distinct types of agents:

- **Strategic Agents (Player 1 and Player 2):** These are autonomous, concurrent entities responsible for decision-making and competitive play.
- **Game Master Agent:** A reactive agent that maintains the environment, validates the moves against the rules, and manages the turn-taking logic by updating the shared game state.

2.3 Specification of Agents (Inputs, Outputs, Task)

2.3.1 Strategic Agents

Each Strategic Agent is an independent, concurrent process responsible for performing high-level reasoning to determine optimal moves.

- **Inputs:** A direct request message from the Game Master and the current 9×9 board state perceived from the shared environment.
- **Outputs:** A specific move coordinate pair (x, y) and a confidence heuristic value.
- **Task:** The primary task is to execute a **Minimax algorithm with Alpha-Beta pruning** to search for the most advantageous move within a limited depth d . Because the outcomes of Ultimate Tic-Tac-Toe are too numerous for a total search, the agent uses a heuristic evaluation function to judge non-terminal positions. Mathematically, the agent aims to maximize its payoff function $V(s)$ in the game tree:

$$V(s) = \begin{cases} +1 & \text{if } s \text{ is a win for Agent 1} \\ -1 & \text{if } s \text{ is a win for Agent 2} \\ 0 & \text{if } s \text{ is a draw} \\ \max_a V(\text{Result}(s, a)) & \text{if it is Agent 1's turn} \\ \min_a V(\text{Result}(s, a)) & \text{if it is Agent 2's turn} \end{cases}$$

2.3.2 Game Master Agent

The Game Master acts as the central coordinator of the system, ensuring the integrity of the game rules.

- **Inputs:** Move coordinates from Strategic Agents and the current legal constraints of the board state.
- **Outputs:** Status updates to the shared board environment and "Turn Request" messages sent to the Strategic Agents.
- **Task:** To validate submitted moves, update the global board state, check for win conditions, and manage the active local board constraint.

The Game Master is classified as a **reactive agent** because its behavior is strictly driven by a stimulus-response loop. It does not engage in proactive planning or internal goal-searching. Instead, it waits for a move submitted by a Strategic Agent and provides a response by validating and updating the environment based on fixed rules.

2.4 Communications Among Agents

Communication is implemented through a Blackboard model, which serves as a centralized shared memory to facilitate coordination without direct message passing between players. This design ensures a decoupled architecture where agents interact solely through a shared data structure.

- **The Blackboard:** A shared data structure containing the current 9×9 grid status and a synchronization flag used to coordinate access.
- **Flow:** The Game Master updates the environment and sets the synchronization flag to signal the active player.

- **Interaction:** The Strategic Agent monitors the Blackboard, reads the board state to compute its move, and writes the resulting coordinates back to the shared memory.
- **Validation:** The Game Master detects this update, validates the move, and updates the synchronization flag to transition the turn to the next agent.